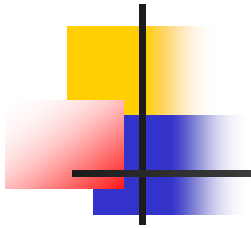
言語・オートマトン
形式言語と文字列の数学

山本章博
国際高等教育院
2026年度版
連絡先: yamamoto.akihiro.5m@kyoto-u.ac.jp



形式言語と文字列の数学

計算機科学における言語の分類

- 自然言語: 日本語，英語，ドイツ語，…
  - 人間が人間に意図を伝える
  - 文構造と意味は人間の知的活動中に成立
    文法は後から構成している
    - 国際補助語 (エスペラント…)
- 人工言語(形式言語)
  - プログラミング言語(C, C++, Java, Scheme,…)
    人間がコンピュータに指示する
  - マークアップ言語(HTML, XML,…)
    コンピュータ間のデータ授受
  - 文構造と意味は設計され，定義されている

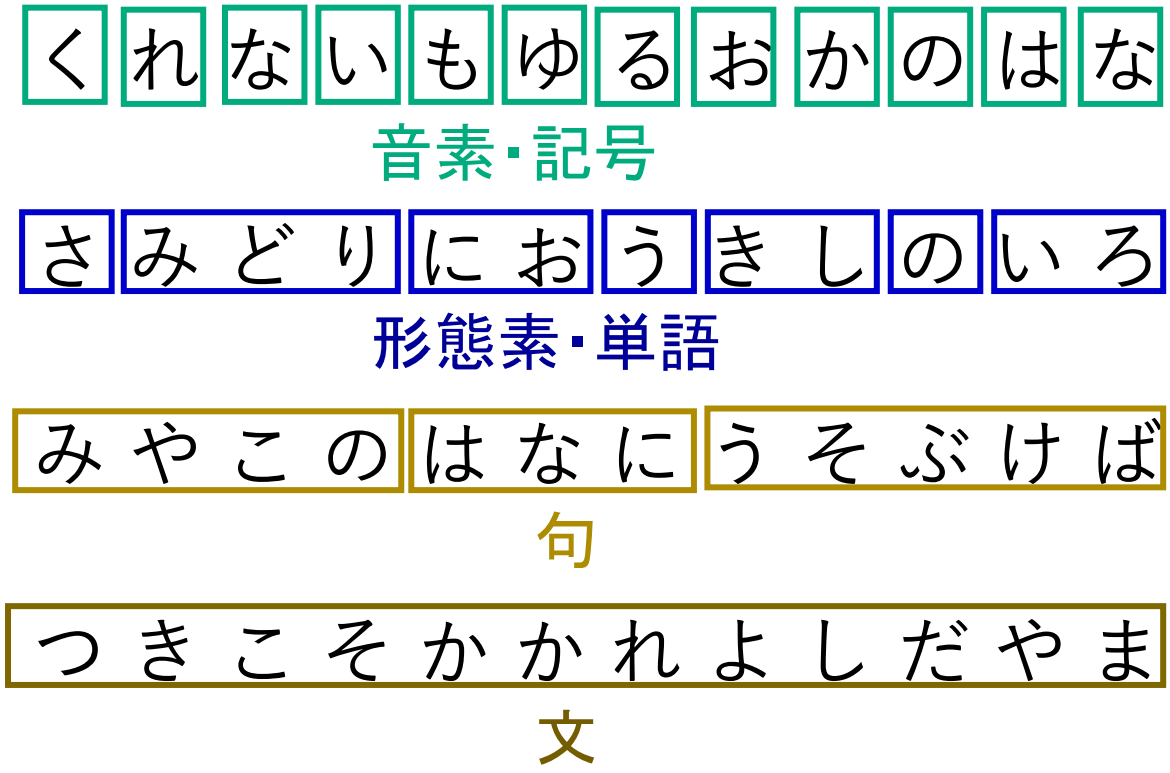
注意

- 計算機科学では, 形式言語もしくは言語を
  ある規則性(文法)にしたがって並べられた
    文字または音素からなる語
    語(形態素)からなる文
  の集合
  と定義する.



- 意味と語や文を切り離す.
  - 文や語の意味を考えない.
  - 文法の機能にだけ注目

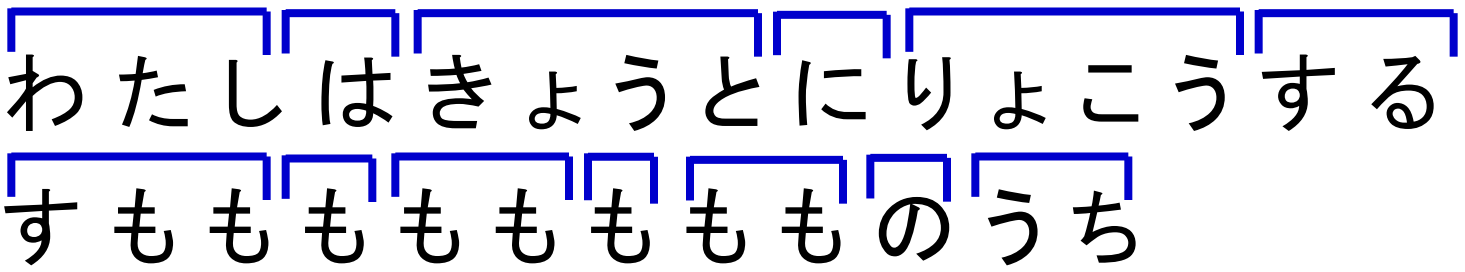
自然言語の文と構造


くれないもゆるおかのはな
音素・記号
さみどりにおうきしのいろ
形態素・単語
みやこのはなにうそぶけば
句
つきこそかかれよしだやま
文

形態素(morpheme)

- 文字列(音素列)において意味を持つ最小単位
  - それ以上分解すると意味をなさなくなる

例



  - 自然言語については，一つの言語に対して，複数の文法が考案されている．

    ⇒ “単語”の定義は文法に依存する

  - 英語でも，接頭辞(un-, non-など)や接尾辞(-tion, -ible)などは一つの形態素として扱う

人工言語の文と構造

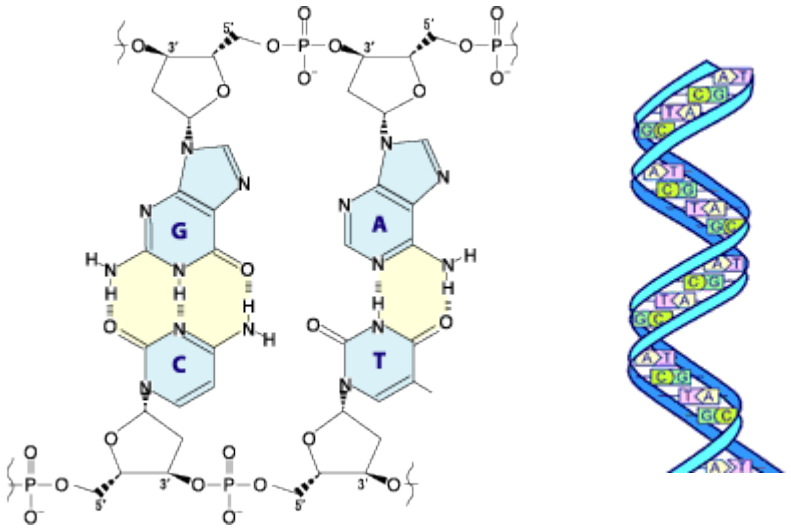
- マークアップ言語，プログラミング言語などでは，処理の最小単位をトークンとよぶ．
  - 最近のLLMでは，自然言語に対しても処理の最小単位をトークンとよんでいる．

例 C言語の場合

```
void  main ( ) {
        int  par ;
        par = par + 12 ;
}
```

DNA

- DNA, RNA配列の類似性



Wikipediaより

…ATTTCACTTGGGTTTAAAATG
CTGTGACCTTGAGTAAGTTGCC
GTCTCTGAATCTGATCCTTTCG
ATTTCCCATTCTCCAAACTGAG
AACTAGCACTGCTGAGACGTGG
TTATTTCCAATAATAATTTGTA
TATTTTACATAACGCACCACAC
CAACATCTTCACCCAGTTGGAG
CCTACTCCTTTGCTCCCGCTG…

2024年のノーベル賞

物理学賞

The Nobel Prize in Physics 2024

John Hopfield

"for foundational discoveries and inventions that enable machine learning with artificial neural networks"



John Hopfield. Ill. Niklas Elmehed © Nobel Prize Outreach

Geoffrey Hinton

"for foundational discoveries and inventions that enable machine learning with artificial neural networks"



Geoffrey Hinton. Ill. Niklas Elmehed © Nobel Prize Outreach

今日の強力な機械学習(AI)の基礎となる手法を開発するための物理学に基づいたツール(計算原理)を開発.

化学賞

The Nobel Prize in Chemistry 2024

David Baker

"for computational protein design"



David Baker. Ill. Niklas Elmehed © Nobel Prize Outreach

Demis Hassabis

"for protein structure prediction"



Demis Hassabis. Ill. Niklas Elmehed © Nobel Prize Outreach

John Jumper

"for protein structure prediction"



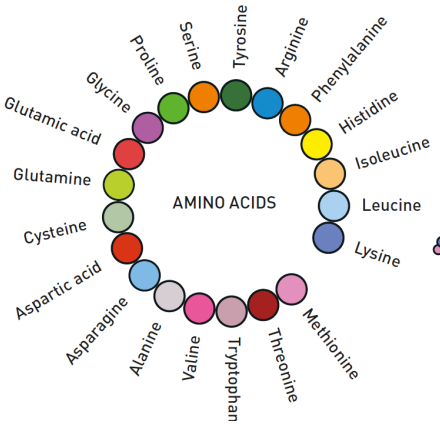
John Jumper. Ill. Niklas Elmehed © Nobel Prize Outreach

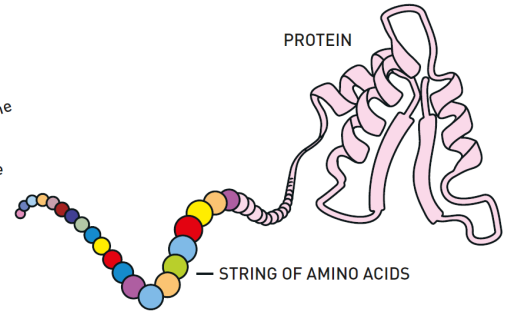
Hassabis氏とJumper氏はタンパク質の複雑な構造を予測するという問題を解決するAIモデルを開発

https://www.nobelprize.org/all-nobel-prizes-2024/

立体構造予測のためのAI

- たんぱく質はアミノ酸の巨大な鎖
- 複雑な立体構造をもつ

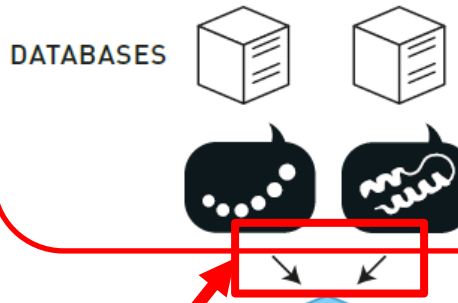


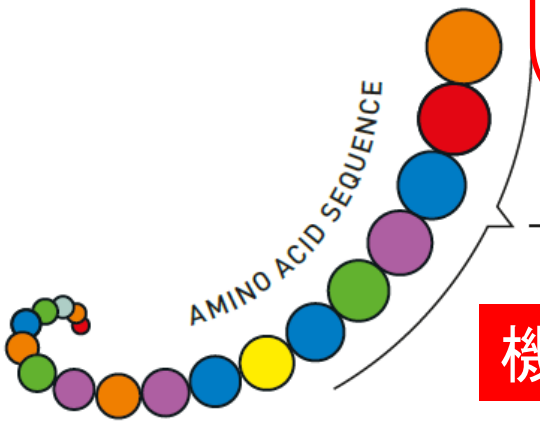


©Johan Jarnestad/The Royal Swedish Academy of Sciences

立体構造が不明なたんぱく質の立体構造を予測

既知のたんぱく質と立体構造の大量データ





AlphaFold2

機械学習

ALYSIS

立体構造を予測!



https://www.nobelprize.org/prizes/chemistry/2024/press-release/

記号列の数学

- 列は計算機科学の基本をなしている
  - 1と0の列
  - ASCII文字の列
  - ひらがな,漢字の列
  - 単語の列
  - A, T, C, Gの列

- 四則演算のような強力な演算がない
  - 連接: $s \cdot t$　記号列 s の後に t をつなげる

    結合律: $s \cdot (t \cdot u) = (s \cdot t) \cdot u$

    単位元: 空の記号列 ε : $\varepsilon \cdot s = s \cdot \varepsilon = s$
  - 演算をアルゴリズムで定義する

記号・文字・アルファベット・語

形式言語理論では

- 操作の最小単位を記号または文字と呼ぶ
  - 化学物質において原子を最小単位とするか，分子を最初単位とするかは状況によって変わる.
  - トークンや形態素の構成を議論するときには，英文字や数字を各英単語を記号もしくは文字として扱う
  - トークンからなる式を議論するときには，トークンを記号もしくは文字として扱う．英単語から構成される文を議論するときには，各英単語を記号もしくは文字として扱う.
- 対象とする記号または文字の有限集合をアルファベット(alphabet)とよぶ
- 有限長の記号からなる列を記号列または語(word)とよぶ

記号列データ

- Σ：アルファベット(alphabet)
- Σ^*：Σ中の記号からなる有限列全体の集合
  - 空列をεで表す
  - $\Sigma^+ = \Sigma^* - \{\varepsilon\}$

Example 1

$\Sigma = \{a, b\}$

$\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots, abaabab, \ldots\}$

Example 2

$\Sigma = \{A, T, C, G\}$

$\Sigma^* = \{\varepsilon, A, T, C, G, AA, AT, AC, AG, \ldots, AAA, AAT, \ldots\}$

列と系列

- 列：string　　系列：sequence
- 部分列(部分語)：substring　(subword)
  部分系列　：subsequence

例　w = abaab

ba, baa などはwの部分列(部分語) であり，
部分系列でもある

aaa や bbはwの部分列(部分語) ではないが，
部分系列である

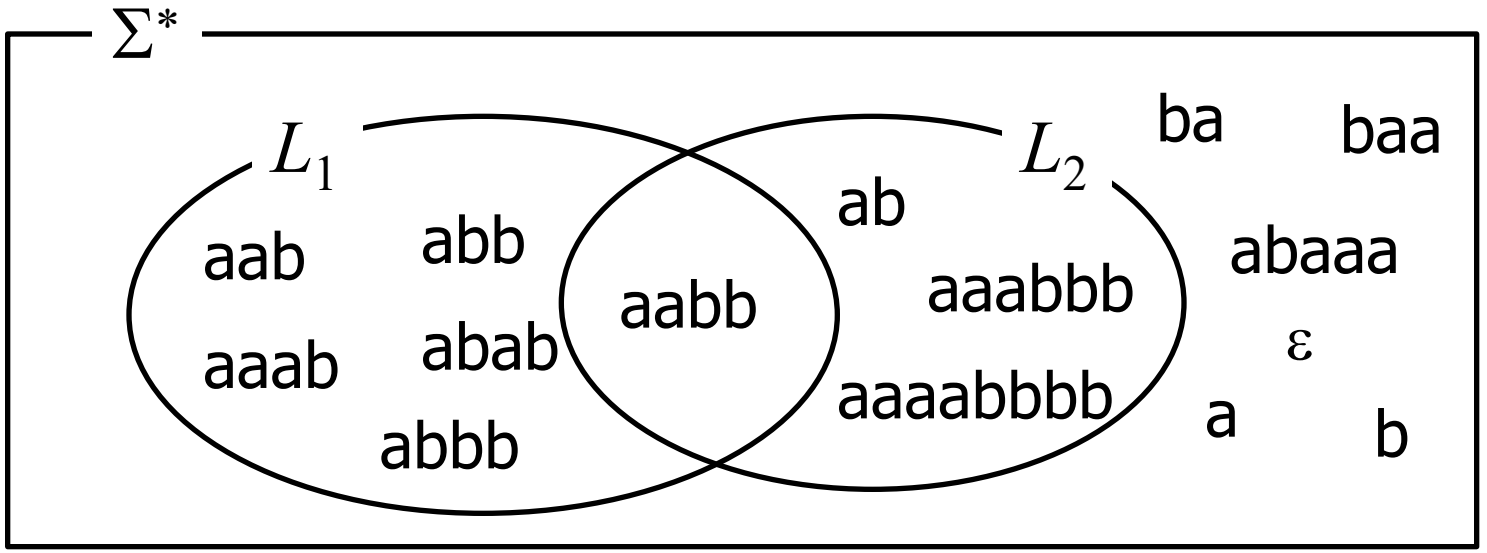
形式言語

- 全体集合をΣ^*とするとき，Σ^*の部分集合を形式言語(あるいは単に言語)とよぶ.
  - 以後，アルファベットΣが文脈から明らかなときは，単に記号列の集合とよぶこともある.

例 $\Sigma = \{a, b\}$, $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$

$L_1 = \{aab, abb, aaab, aabb, abab, abbb, \ldots\}$

$L_2 = \{ab, aabb, aaabbb, aaaabbbb, \ldots\}$



集合の内包的表現と規則性

集合を表す2種類の方法

- 要素を並べて表す(外延的)

$$\Sigma = \{a, b\}$$

$$L = \{ab, aab, abab, aaab, abaab, aaaabbbb, \ldots\}$$

- 全体集合の要素がその集合に属する条件を用いて表す(内包的)

$$L = \{w \in \Sigma^* \mid w \textit{ starts with } a \textit{ and ends with } b\}$$

形式言語と規則(1)

例1 $\Sigma = \{0, 1, 2, \ldots, 9\}$　　(注) Σ の要素は数字

$L = \{ x \mid x$ は自然数の正しい10進法表記 $\}$

$3509 \in L$, $00427 \notin L$

例2 $\Sigma = \{0, 1, 2, \ldots, 9, .\}$

$L = \{ x \mid x$ は有限小数の正しい10進法表記 $\}$

$3509.1 \in L$, $42.7.8 \notin L$

$3509.10 \in L$?　　$.1078 \in L$?

形式言語と規則(2)

- 言語である以上，正しい“語”または“文”を定める規則がある.

例 あるプログラミング言語の変数

例 XML言語のタグ

例 日本語単語の綴りにおける“ん”の使い方

例 英単語の綴りにおけるqの使い方

受理系と生成系

- 言語である以上，正しい“語”または“文”を定める規則がある.
- 規則の表現方法には大きく分けて受理系と生成系がある.
  - 受理系：受け取った“語”または“文”が正しいかどうかを判別する手法
  - 生成系：正しい“語”または“文”を生成する際に遵守する規則

受理系(受理機械)	生成系(生成文法)
Turing Machine	句構造文法(0型)
線形有界オートマトン	文脈依存文法(1型)
非決定性プッシュダウン・オートマトン	文脈自由文法
有限状態オートマトン	正則文法

[余談] 現代のAIにも認識系(画像や音声の認識)と生成(合成)系(画像や音声の合成)がある.

用語と記号(1)

アルファベット Σ を一つ固定する.

- 記号列 $w = c_1 c_2 \cdots c_n$ の長さを $|w| = n$ と定義する.
  空列εに対しては $|\varepsilon| = 0$ と定義する.
  例 $|\mathrm{abb}| = 3, |\mathrm{abaabab}| = 7$
- 形式言語 L と自然数 n に対して形式言語$L|_n$を
  $L|_n = \{w \in \Sigma^* \mid w \in L$ かつ $|w| = n\}$ と定義する.
- 形式言語 L, M に対して形式言語 $L \cdot M$ を
  $L \cdot M = \{ uv \in \Sigma^* \mid u \in L$ かつ $v \in M\}$と定義する.
  - 演算子・はしばしば省略することがある.

  例 $L = \{\mathrm{ab}, \mathrm{abb}, \mathrm{ba}\}$, $M = \{\varepsilon, \mathrm{bb}, \mathrm{bab}\}$ とするとき
  $L \cdot M = \{\mathrm{ab}, \mathrm{abb}, \mathrm{ba}, \mathrm{abbb}, \mathrm{abbbb}, \mathrm{babb}, \mathrm{ba}, \mathrm{babb}, \mathrm{babab}\}$.

命題と注意

命題　アルファベット Σ を一つ固定する.

形式言語 L, M, N に対して $(L \cdot M) \cdot N = L \cdot (M \cdot N)$.

したがって，$(L \cdot M) \cdot N$ を $L \cdot M \cdot N$ と表記する.

注意　一般には　$L \cdot M \neq M \cdot N$

命題の証明

- $(LM)N \subseteq L(MN)$ と $(LM)N \supseteq L(MN)$ の両方を示す

$(LM)N \subseteq L(MN)$の証明

任意の $x \in (LM)N$ は集合$(LM)N$の定義から,

$x = yz$ ($y \in LM$ かつ $z \in N$) と分解できる.

さらに 集合LMの定義から, $y = uv$ ($u \in L$ かつ $v \in M$)と分解できる.

つまり, $x = (uv)z$ である.

連接演算の結合律から $x = (uv)z = u(vz)$ が成立つ.

ここで $vz \in MN$ であるから, $x = u(vz) \in L(MN)$ である.

任意の $x \in (LM)N$ について $x = \in L(MN)$ なので $(LM)N \subseteq L(MN)$.

$(LM)N \supseteq L(MN)$の証明も同様である.

用語と記号(2)

- 形式言語 L と非負整数n に対して形式言語L^n を以下のように帰納的に(再帰的に)定義する.
  - $L^0 = \{\varepsilon\}$. $(L^0 \neq \emptyset$に注意$)$
  - $L^1 = L$.
  - $L^{n+1} = L(L^n)$.

注意1. 形式言語理論では L^nはLのn回の直積 $L \times L \times \cdots \times L$ではない.

注意2. $L(L^n) = (L^n)L$ が成り立つ.

例 $L = \{$ab, abb, ba$\}$, $M = \{\varepsilon$, bb, bab$\}$ とするとき

$L^2 = \{$abab, ababb, abba, abbab, abbabb, abbba,
baab, baabb, baba$\}$,

$M^3 = \{\varepsilon$, bb, bab, bbbb, bbbab, babbab, babbb, babbab,
bbbbbab, bbbabbab, bbbabbb, bbbabbab
babbbbab, babbabbab, babbabbb, babbabbab$\}$

用語と記号(3)

- 形式言語 L に対して L^* を

$$L^* = L^0 \cup L^1 \cup \ldots \cup L^n \cup \ldots$$

$$= \{w \in \Sigma^* \mid \text{ある非負整数} n \text{ が存在して } w \in L^n\}$$

と定義する.

例 $L = \{\text{ab, abb}\}$ とするとき

$L^* = \{\varepsilon$, ab, abb, abab, ababb, abbab, abbabb,
ababab, abababb, ababbab, ababbabb,
abbabab, abbababb, abbabbab, abbabbabb,
abababab, ababababb, abababbab, abababbabb,
ababbabab, ababbababb, ababbabbab, ababbabbabb
abbababab, abbababbb, abbababbab, abbababbabb,
abbabbabab, abbabbababb, abbabbabbab, abbabbabbabb , …$\}$

注意

- 計算機科学では，形式言語もしくは言語を
  ある規則性(文法)にしたがって並べられた
  文字または音素からなる語
  語(形態素)からなる文
  の集合
  と定義する.
  - 目的に応じて，文字または音素を最小単位とする場合と語(形態素)を最小単位とする場合がある.



- 意味と語や文を切り離す.
  - 文や語の意味を考えない.
  - 文法の機能にだけ注目



帰納的な定義と証明

課題 3

アルファベットを Σ とするとき, 文字列 $w \in \Sigma^*$ を逆順に並べて得られる文字列を w^R と表すことにする. 例えば $(\mathrm{abbaaa})^R = \mathrm{aaabba}$, $(\mathrm{abcabcba})^R = \mathrm{abcbacba}$ である. 形式言語

$$L_{Pal} = \{w \in \Sigma^* \mid w = w^R\}$$

の要素を回文 **(palindrome)** という. 以下では $\Sigma = \{\mathrm{a},\mathrm{b}\}$ とする.

(a) 演算 ^{R} を文字列の構成 (あるいは長さ) に関して帰納的に定義せよ.

(b) 以下の等式が成り立つことを文字列の構成 (あるいは長さ) に関する帰納法を用いて示せ.

$$L_{Pal} \quad = \quad \{u(u^R) \mid u \in \Sigma^*\} \cup \{u\mathrm{a}(u^R) \mid u \in \Sigma^*\} \cup \{u\mathrm{b}(u^R) \mid u \in \Sigma^*\}.$$

(c) $\sharp(L_{Pal}|_{2n-1}) = \sharp(L_{Pal}|_{2n})$ が成り立つことを示せ.

数学的帰納法

例 すべての0でない自然数$n \in \mathbf{N}^+$について

$$a_1 = 2, \quad a_n = 2 - \frac{1}{a_n}$$

と定義するとき

$$a_n = \frac{n+1}{n} \quad \cdots\cdots①$$

であることを数学的帰納法で証明する

I. $n=1$ のときは $a_1 = \frac{1+1}{1} = 2$

II. $n=k$ のときに ①が成り立つと仮定して$n=k+1$ のとき

$$a_{k+1} = 2 - \frac{1}{a_k} = 2 - \frac{k}{k+1} = \frac{k+2}{k+1} = \frac{(k+1)+1}{k+1}$$

なので①は成り立つ

I,IIより，すべての0でない自然数$n \in \mathbf{N}^+$について①は成り立つ

数学的帰納法の根拠

- 数学的帰納法は，自然数に関する性質(公理)が根拠
  - I. 0(または1)は自然数である.
  - II. nが自然数であれば，$n+1$も自然数である.
  - III. I, IIで自然数とわかるものだけが自然数である.

(注)IIIがなければ，次のような集合が自然数全体の集合ではないとはいえなくなる

$$\{-1, 0, 1, 2, 3, 4, 5,...\}$$

$$\{0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5,...\}$$

(注)AI・機械学習では「帰納法(induction)」を

個々の具体的な事例から一般に通用するような原理・法則

などを導き出すこと

[デジタル大辞泉 https://www.weblio.jp/content/帰納]

という科学哲学用語としての意味で使う.

帰納的な定義と証明

例 すべての0でない自然数$n \in \mathbf{N}^+$について

$$a_1 = 2, \quad a_n = 2 - \frac{1}{a_n}$$

と定義するとき

帰納的な定義
(漸化式)

$$a_n = \frac{n+1}{n} \quad \cdots\cdots①$$

であることを数学的帰納法で証明する

帰納的な証明

I. $n=1$ のときは $a_1 = \frac{1+1}{1} = 2$

II. $n=k$ のときに ①が成り立つと仮定して$n=k+1$ のとき

$$a_{k+1} = 2 - \frac{1}{a_k} = 2 - \frac{k}{k+1} = \frac{k+2}{k+1} = \frac{(k+1)+1}{k+1}$$

なので①は成り立つ

I,IIより，すべての0でない自然数$n \in \mathbf{N}^+$について①は成り立つ

記号列(文字列)と帰納法

- アルファベットSの要素である記号(文字)からなる有限の記号列についても以下のように定義する

  I. 空列ε(または任意の記号$c \in \Sigma$)は記号列である.

  II. wが記号列であれば, 任意の$c \in \Sigma$に対して

  (a) wcも記号列である.

  (b) cwも記号列である.

  III. I, IIで記号列とわかるものだけが記号列である.

(注) (a), (b)のいずれを採用するかは, 何を定義したいのか, 何を証明したいのかによる.

(注) アルファベットS a としたときの記号列を自然数の1進法表記と解釈すれば, 自然数の帰納的定義となる.

記号列に対する関数の帰納的な定義

- 数列 $a_1, a_2, \ldots, a_n, \ldots$ を関数a: $\mathbf{N}^+ \to \mathbf{R}$ とみなすとき，関数aを帰納的に(漸化式によって)定義することができる.

$$a(1) = 2, \quad a(n+1) = 2 - \frac{1}{a(n)}$$

- 同様に，記号列に対する関数 f: $\Sigma^* \to \Sigma^*$やf: $\Sigma^* \to$ **R**なども帰納的に(漸化式)によって定義することができる.
- 例 記号列 $c_1c_2\cdots c_n$を$c_n\cdots c_2c_1$のように逆順に並べる関数Rを帰納的に定義すると

$$\mathrm{R}(\varepsilon) = \varepsilon, \quad \mathrm{R}(cw) = \mathrm{R}(w)c$$

実際

$$\mathrm{R}(abaa) = \mathrm{R}(baa)a = (\mathrm{R}(aa)b)a = ((\mathrm{R}(a)a)b)a$$
$$= (((\mathrm{R}(\varepsilon)a)a)b)a = ((((\varepsilon)a)a)b)a = aaba$$

記号列に対する定理の帰納的な証明(1)

補題1 任意の記号$c\in\Sigma$と記号列 $w\in\Sigma^*$ に対して $\mathrm{R}(wc)=c\mathrm{R}(w)$

証明 I. $w=\varepsilon$ のとき, $\mathrm{R}(\varepsilon c)=\mathrm{R}(c\varepsilon)=\mathrm{R}(\varepsilon)c=\varepsilon c=c\varepsilon=c\mathrm{R}(\varepsilon)$ なので
補題は成立している

II. 記号列 $w=u$ に対して補題が成立していると仮定して
$w=du$ のときにも補題が成立していることを示す.
任意の$c\in\Sigma$に対して

$\mathrm{R}(duc)=\mathrm{R}(uc)d$ Rの定義から
$=(c\mathrm{R}(u))d$ 帰納法の仮定から
$=c(\mathrm{R}(u)d)$ 結合律
$=c\mathrm{R}(du)$ Rの定義の右辺と左辺 を入替え

I, IIから補題が成り立つ

記号列の長さに関する帰納法にするときは「長さnの」を入れる

記号列に対する定理の帰納的な証明(2)

補題2 $\mathrm{R}(\mathrm{R}(w)) = w$

証明 I. $w = \varepsilon$ のとき, Rの定義から$\mathrm{R}(\mathrm{R}(\varepsilon))= \mathrm{R}(\varepsilon)=\varepsilon$ なので
補題は成立している

II. 記号列 w に対して補題が成立していると仮定する.
任意の$c \in \Sigma$に対してRの定義と補題1から

$$\mathrm{R}(\mathrm{R}(cw)) = \mathrm{R}(\mathrm{R}(w)c) = \mathrm{R}(\mathrm{R}(w)c) = c\mathrm{R}(\mathrm{R}(w))$$

帰納法の仮定から $\mathrm{R}(\mathrm{R}(w)) = w$なので $\mathrm{R}(\mathrm{R}(cw)) = cw$

I, IIから補題が成り立つ

記号列に対する定理の帰納的な証明(2)

補題2 $\mathrm{R}(uv) = \mathrm{R}(v)\mathrm{R}(u)$

証明 記号列uに関する帰納法を用いる

I. $u = \varepsilon$ のとき, $\mathrm{R}(\varepsilon v) = \mathrm{R}(v) = \mathrm{R}(v)\varepsilon = \mathrm{R}(v)\mathrm{R}(\varepsilon)$ なので
補題は成立している

II. 記号列 u に対して補題が成立していると仮定する.
任意の$c \in \Sigma$に対して

$$\begin{aligned} \mathrm{R}((cu)v) = \mathrm{R}(cuv) &= \mathrm{R}(uv)c && \text{Rの定義から} \\ &= (\mathrm{R}(v)\mathrm{R}(u))c && \text{帰納法の仮定} \\ &= \mathrm{R}(v)(\mathrm{R}(u)c) && \text{結合律} \\ &= \mathrm{R}(v)\mathrm{R}(cu) && \text{Rの定義から} \end{aligned}$$

I, IIから補題が成り立つ